STM32 Mobile Chassis Development Manual

Large ROS Research Robot Series

English Technical Edition

Revision History

Version	Date	Description
V1.0	2024-08-28	Initial release

Technical reference for ROS mobile robot development and integration.

Preface

Welcome to the STM32 Mobile Chassis Development Manual. This manual provides practical guidance for developing and integrating large ROS research robot platforms. It is intended for robotics enthusiasts, engineering students, educators, and professional researchers who need a clear understanding of the chassis hardware, control structure, kinematics, and development workflow.

The manual introduces the fundamental principles and composition of a ROS-enabled mobile chassis, and then covers kinematic analysis, communication interfaces, motor control, control modes, hardware architecture, firmware organization, and firmware programming methods. The objective is to help readers connect the underlying theory with practical implementation and build reliable mobile robot systems.

The content applies to several wheeled mobile robot configurations, including differential-drive, Ackermann-steering, Mecanum-wheel, omni-wheel, and four-wheel-drive (4WD) chassis. Each configuration has different motion characteristics and hardware requirements.

For best use of this manual, readers should have basic knowledge of robotics, ROS concepts, programming (especially C), and basic electronics/electrical engineering. Additional ROS/ROS 2 and Ubuntu tutorials may be required for higher-level application development.

Contents

1. Introduction
- 1.1 Relationship Between the Mobile Chassis and ROS
- 1.2 Chassis Coordinate Convention

2. Mobile Chassis Composition and Description
- 2.1 Controller and Motor Drivers
- 2.2 Encoders, Motors, and Steering Servo
- 2.3 Wheels
- 2.4 OLED Display Data
- 2.5 Switches and Buttons
- 2.6 IMU Zero-Bias Update
- 2.7 Potentiometer Functions
- 2.8 LED and Buzzer
- 2.9 LED Strip
- 2.10 Bluetooth App Debugging

3. Chassis Types and Kinematic Analysis
- 3.1 Kinematic Analysis

4. Chassis Motor Control
- 4.1 Wheel Linear-Speed Calculation
- 4.2 Incremental PI Controller

5. Chassis Control Modes

6. Chassis System Architecture and Control Flow
- 6.1 Robot Hardware Architecture
- 6.2 STM32 Firmware Architecture
- 6.3 Motor Control Flow

7. Firmware Programming
- 7.1 Serial/USB Programming
- 7.2 SWD Programming

8. Notes and Precautions
- 8.1 Code and RTOS Tasks
- 8.2 Power Interfaces
- 8.3 Motor Use and Protection
- 8.4 Battery Use and Maintenance

1. Introduction

This chapter explains how the Robot Operating System (ROS) works together with the mobile chassis and introduces the coordinate convention used by the chassis.

1.1 Relationship Between the Mobile Chassis and ROS

ROS provides middleware that enables efficient communication and coordination among hardware and software components. Within this framework, the mobile chassis is not only the physical mobility platform but also a key communication endpoint that exchanges data with the ROS environment through a serial interface.

The main roles of the mobile chassis in a ROS system are:

- Communication node: the chassis interface participates in the ROS network and sends/receives robot-motion-related messages.
- Data integration: the chassis provides real-time state feedback such as wheel odometry and IMU data. These data are important for localization, navigation, and motion planning.
- Control execution: the chassis receives commands such as target velocity and steering commands, converts higher-level control objectives into motor drive signals, and executes the required motion.

This modular architecture allows the chassis to integrate with other ROS nodes. As an application evolves, higher-level nodes can be added or replaced without redesigning the low-level chassis control system.

The STM32 chassis described in this manual includes the low-level communication structure, data handling, packet processing concepts, motor/encoder control, and the interface between the chassis controller and the host computer running ROS or other software.

1.2 Chassis Coordinate Convention

Coordinate conventions are fundamental for describing position, orientation, and motion. The chassis uses a standard planar robot convention: the positive X-axis points forward, the positive Y-axis points to the left, and positive yaw about the Z-axis is counter-clockwise.

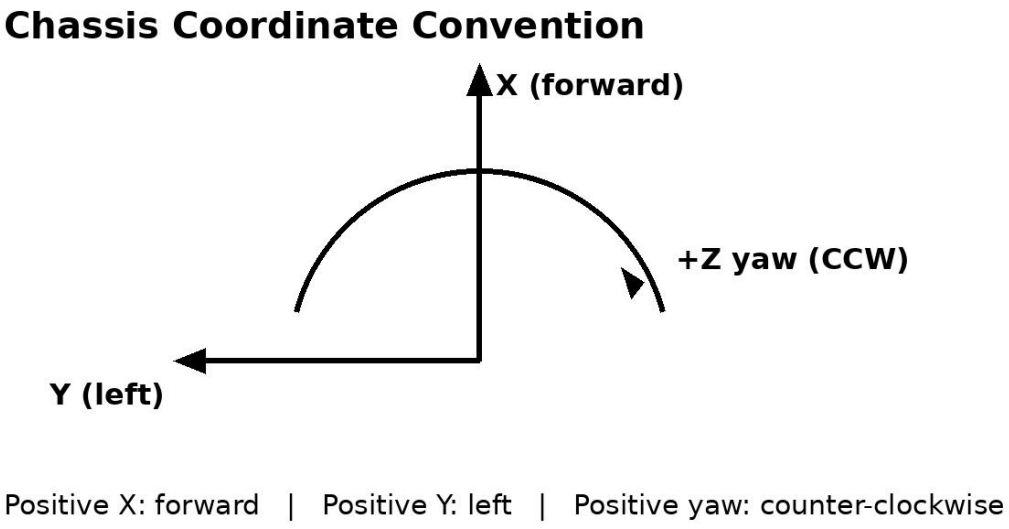


Figure 1-1. Chassis coordinate convention

2. Mobile Chassis Composition and Description

A robot system can be divided into three functional layers. The upper decision layer requires comparatively high computing power but is less time-critical and normally runs on a computer operating system. The perception layer uses sensors such as LiDAR, depth cameras, and inertial sensors to measure the surrounding environment. The low-level execution layer has lower computational demand but strict real-time requirements; it is typically implemented on a microcontroller in C and is responsible for executing motion commands and reporting robot status.

The mobile chassis is part of this low-level execution layer. Typical hardware includes a controller board, motor drive stage, encoder-equipped motors, wheels, and - for Ackermann configurations - a steering servo. Hardware details vary across chassis versions.

2.1 Controller and Motor Drivers

Controller

The main controller is based on the STM32F407VET6 MCU. It receives commands from the host computer and converts them into the corresponding low-level control signals. The controller is designed for stable, real-time operation and also integrates an ICM20948 six-axis IMU for motion and attitude feedback.

The controller exposes multiple interfaces for robot integration and expansion, including serial interfaces, CAN, encoder inputs, IMU communication, and peripheral control. Depending on the chassis revision, the motor drive stage may be implemented with an external dual-channel motor driver board or integrated with the controller hardware.

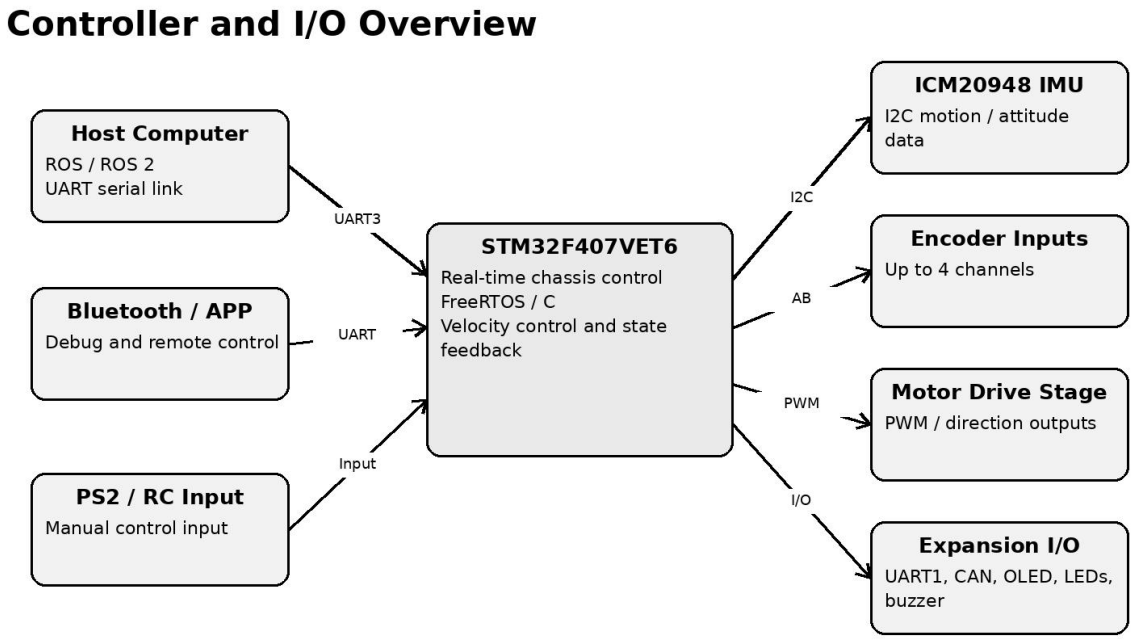


Figure 2-1. Controller and I/O overview

Motor Drive Stage

The motor driver is the interface between the STM32 controller and the DC motors. It converts low-power control signals into the voltage and current required by the motors, allowing the motors to run at the commanded speed and direction.

Some chassis variants use a dual-channel D50A motor driver board connected to the STM32 controller by keyed signal cables. The keyed connectors help prevent reverse or incorrect insertion. The number of driver boards and the wiring method depend on the chassis configuration and the number of drive motors.

Expansion and Wiring Notes

The controller board provides multiple connectors, switches, and peripheral interfaces for expansion. It includes an ICM20948 IMU and up to four encoder input channels. When integrating a third-party motor or encoder, verify the connector pinout before power-up. Refer to the electrical schematic and the hardware troubleshooting materials supplied with the platform.

Motor-to-driver wiring can differ among differential-drive, Mecanum, 4WD, omni-wheel, and Ackermann variants. Use the wiring diagram corresponding to the actual chassis configuration in the supplied hardware documentation.

2.2 Encoders, Motors, and Steering Servo

This section summarizes the basic operating principles of the encoders, drive motors, and steering servo used in the mobile chassis. More detailed motor-control tutorials can be found in the accompanying low-level development materials.

Encoders

Encoders measure and report motor speed and position information. Common encoder types used on mobile robot motors include Hall encoders and GMR (giant magnetoresistance) encoders. Encoder feedback is essential for closed-loop speed control because it allows the controller to compare the measured motor speed with the commanded value.

A Hall encoder typically consists of a magnetic code disk and Hall sensing elements. As the motor rotates, the sensor produces pulse signals. Two square-wave channels, A and B, are phase-shifted relative to one another so that the controller can determine both pulse count and rotation direction.

A GMR encoder uses the magnetoresistance effect to measure displacement. Typical signals include A, B, and Z. Channels A and B provide quadrature position information, while Z can mark a once-per-revolution index position. In the documented configuration, only channels A and B are used. GMR encoders provide high resolution, good accuracy, and low temperature drift.

Encoder resolution is related to the number of counts or lines per revolution. For example, a 13-line Hall encoder generates 13 pulse periods per revolution, while a 500-line GMR encoder generates 500. The phase relationship between A and B is used to determine the direction of rotation.

Motors

The drive motor converts electrical energy into mechanical motion. The chassis uses DC geared motors with fast response and closed-loop controllability. When an encoder is installed on the motor and connected to the controller, motor speed and position can be measured in real time for precise speed and position regulation.

The gear ratio describes the relationship between the motor rotor speed and the output shaft speed. For example, a 1:30 reduction ratio means the motor rotor turns approximately 30 revolutions for one revolution of the output shaft. The reduction gearbox trades speed for increased output torque.

Steering Servo

Ackermann-steering chassis variants use a steering servo to set the steering angle. A servo typically integrates a DC motor, reduction gearing, a drive circuit, and a position-feedback potentiometer in one unit. The drive circuit receives the steering command, the gear train reduces

speed and increases output torque, and the potentiometer provides position feedback so the servo can move to and hold the target angle.

2.3 Wheels

The wheels are the robot chassis contact points with the ground and have a direct effect on mobility, stability, load capacity, traction, and kinematic behavior. Different wheel and chassis structures support different motion patterns, so wheel parameters are also used in the kinematic calculations and wheel-speed conversion described later in this manual.

2.4 OLED Display Data

The controller supports an OLED display for real-time chassis status. The exact layout varies by chassis type, but the displayed information generally includes the current chassis type, IMU/gyro information, target and measured wheel velocities, motor PWM or steering-servo values, active control mode, and battery voltage.

Display Item	Typical Meaning	Applicable Configurations
TYPE	Selected chassis model / mode	All
GYRO / BIAS	IMU or gyro zero-bias / yaw-related status	All
Wheel target / actual speed	Commanded and measured wheel linear velocity	Drive wheels
PWM	Motor output command	Differential / omni / Mecanum / 4WD
SERVO	Steering-servo command / status	Ackermann
Control mode	CAN, APP, PS2, RC, USART, or ROS	All
Battery voltage	Current supply/battery voltage	All

Control-mode abbreviations shown on the OLED include CAN, APP, PS2, R-C, USART, and ROS.

2.5 Switches and Buttons

Enable Switch

The motor enable switch is located on the controller board. The motors can be controlled only when the switch is in the ON state. For startup stability, the firmware forces the motors to remain disabled during the first 10 seconds after power-up. During this period the display may show OFF even if the physical switch is ON, and Serial Port 1 and Serial Port 3 do not accept command data. After initialization, the enable state is updated automatically.

Emergency Stop Interface

An emergency-stop interface is provided on the controller board. Some chassis models have a software emergency-stop switch mounted on the chassis. Availability depends on the mechanical configuration.

USER Button

The USER button is located on the controller board. Double-clicking the USER button updates the gyroscope zero point on supported chassis variants.

RESET Button

Pressing the RESET button restarts the STM32 controller.

2.6 IMU Zero-Bias Update

During power-on initialization, the robot performs IMU zero-bias calibration. If significant map drift is observed during navigation or mapping, manually updating the gyroscope zero point may

improve performance. Keep the robot stationary, then double-click the USER button to perform the update.

2.7 Potentiometer Functions

The controller includes two potentiometers with different functions.

Chassis-Type Selection Potentiometer

Different hardware variants of the same chassis family may use different parameters. A chassis-selection potentiometer is therefore used to select the appropriate vehicle configuration. The current selection can be checked using the TYPE value on the OLED. After changing the TYPE selection, power-cycle the chassis for the new setting to take effect.

Steering-Servo Center Adjustment

Ackermann chassis variants include a potentiometer used to adjust the steering-servo center value. If the servo does not return to the mechanical straight-ahead position after power-up, adjust the center value according to the Ackermann chassis troubleshooting instructions supplied with the platform.

2.8 LED and Buzzer

Buzzer

The controller includes a buzzer. The buzzer must be connected to the designated D11 and BP pins to operate. The robot produces a short beep at power-up. When the voltage falls below approximately 20 V, the buzzer emits a short warning tone at about 1 Hz.

User LED

During the first 10 seconds after power-up, the firmware initializes the chassis and performs IMU zero-drift sampling and calibration. The user LED flashes rapidly during initialization and changes to a normal flash pattern when initialization is complete.

2.9 LED Strip

Some chassis variants include an LED light strip. The LED strip uses the following status meanings:

- Power-on self-test: red, green, and blue alternate.
- Red: automatic charging in progress.
- Green: automatic charging completed / battery full.
- Purple: searching for the charging dock.
- Flashing purple: low battery.
- Yellow: an ultrasonic sensor has detected an obstacle.
- Default operating state: LED strip off.

2.10 Bluetooth App Debugging

When the OLED is not convenient to view, chassis status can be checked through the Bluetooth app debugging function. Connect to the Bluetooth module configured for the platform; the default pairing code is 1234. After connection, the data in the app debug page should begin updating.

Open the debug page and send the command LOG3 to activate debug mode. The returned information can be used to check which chassis functions are enabled and whether the associated hardware is operating normally. If an unknown or unresolved error occurs, contact technical support.

3. Chassis Types and Kinematic Analysis

Kinematic analysis describes how individual wheel velocities relate to the overall translational and rotational motion of the chassis. In normal operation, the host computer sends target planar velocity commands to the chassis; the low-level controller then converts these commands into target wheel velocities and drives the motors accordingly.

Chassis Type	Forward/Reverse	Lateral Motion	In-Place Rotation	Relative Load Capacity	Relative Steering Friction
Mecanum	Yes	Yes	Yes	Lower	Lower
Omni-wheel	Yes	Yes	Yes	Lower	Lower
Differential drive	Yes	No	Yes	Lower	Lower
4WD skid-steer	Yes	No	Yes	Higher	Higher
Ackermann steering	Yes	No	No	Higher	Lower

The process of converting chassis target velocity into wheel target velocity is referred to here as kinematic analysis. It includes two directions:

- Forward kinematics: calculate chassis X, Y, and Z/yaw motion from the individual wheel velocities.
- Inverse kinematics: calculate the individual wheel velocities from the desired chassis X, Y, and Z/yaw motion.

3.1 Kinematic Analysis

Detailed derivations for the supported chassis types are provided in the companion chassis-control and kinematics materials supplied with the development package. The appropriate model should be selected for the actual drive configuration.

4. Chassis Motor Control

Closed-loop motor control is used so that the chassis can accurately respond to velocity commands and maintain stable operation. After inverse kinematics produces a target speed for each wheel, the motor controller uses encoder feedback to regulate the actual speed toward the target.

The firmware uses PID-based speed control. PID refers to proportional, integral, and derivative action. The implementation uses an incremental PI speed controller for the wheel motors.

4.1 Wheel Linear-Speed Calculation

Encoder pulse counts are used to estimate motor speed and then calculate the corresponding wheel linear speed. The following relationship is used:

```
Linear speed = encoder counts per second x wheel circumference / encoder resolution / motor reduction ratio
```

The corresponding C implementation is:

```
MOTOR_A.Encoder = Encoder_A_pr * CONTROL_FREQUENCY * Wheel_perimeter / Encoder_precision;
```

where:

- Encoder_A_pr: raw encoder count data.
- CONTROL_FREQUENCY: control-loop frequency in Hz.
- Encoder_precision: effective encoder resolution, determined by the motor mechanical structure and encoder device.
- Wheel_perimeter: wheel circumference in meters.
- MOTOR_A.Encoder: calculated real-time linear speed of wheel A in m/s.

The same method is applied to motors B, C, and D as applicable.

4.2 Incremental PI Controller

The target motor speed from kinematics and the measured motor speed from the encoder are fed to the speed controller. The controller adjusts the PWM command so the measured speed approaches the target speed. The firmware example uses the following incremental PI implementation:

```
/**************************************************************************
* Function: Incremental PI speed controller
* Input:    Encoder – measured encoder/speed value
*           Target  – target speed
* Return:   Pwm     – motor PWM command
* Formula:  pwm += Kp[e(k)–e(k–1)] + Ki*e(k)
**************************************************************************/
int Incremental_PI_A(float Encoder, float Target)
{
    static float Bias, Pwm, Last_bias;
    Bias = Target – Encoder;
    Pwm += Velocity_KP * (Bias – Last_bias) + Velocity_KI * Bias;

    if (Pwm > 7200)  Pwm = 7200;
    if (Pwm < –7200) Pwm = –7200;

    Last_bias = Bias;
    return Pwm;
}
```

This manual does not expand on PID theory in detail. Additional PID and motor-control material is available in the companion low-level motor-control tutorials.

5. Chassis Control Modes

The chassis supports multiple command sources, including Bluetooth app control, PS2 controller input, RC transmitter input, CAN control, and serial control. Serial control includes a ROS mode and a general USART mode. After startup, the active control mode is shown on the OLED, and the default mode is ROS serial control.

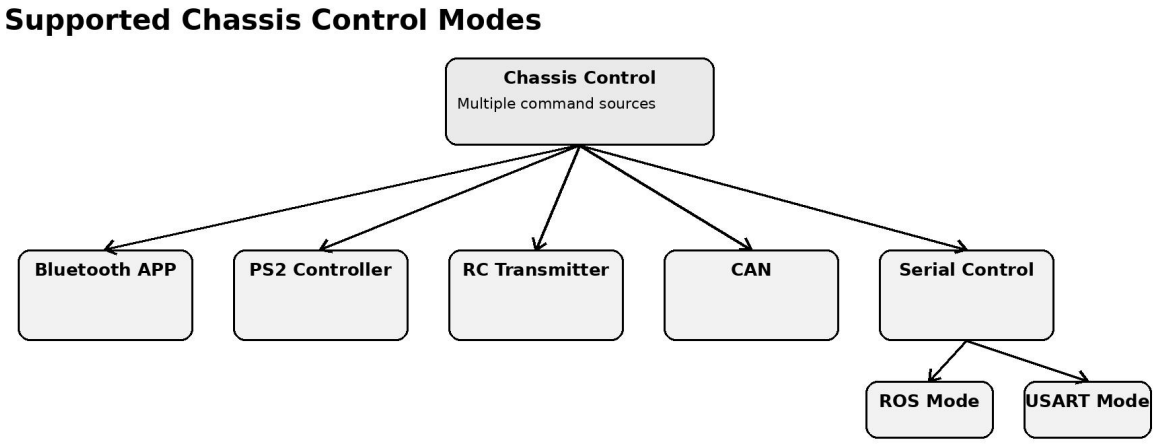


Figure 5-1. Chassis control modes

Detailed operation of each control mode is provided in the corresponding control tutorials within the supplied development package.

6. Chassis System Architecture and Control Flow

The hardware and software components can be summarized as a layered control architecture. A host or onboard computer runs ROS/ROS 2 applications, sensors provide environmental data, and the STM32 controller executes real-time motion control while handling motor, encoder, and low-level I/O functions.

6.1 Robot Hardware Architecture

Typical system components include a Raspberry Pi or NVIDIA Jetson Orin Nano class onboard computer, LiDAR, the STM32 controller, motors, encoders, a motor driver stage, Bluetooth, PS2/RC inputs, and the IMU. The controller also reserves Serial Port 1 and CAN for expansion.

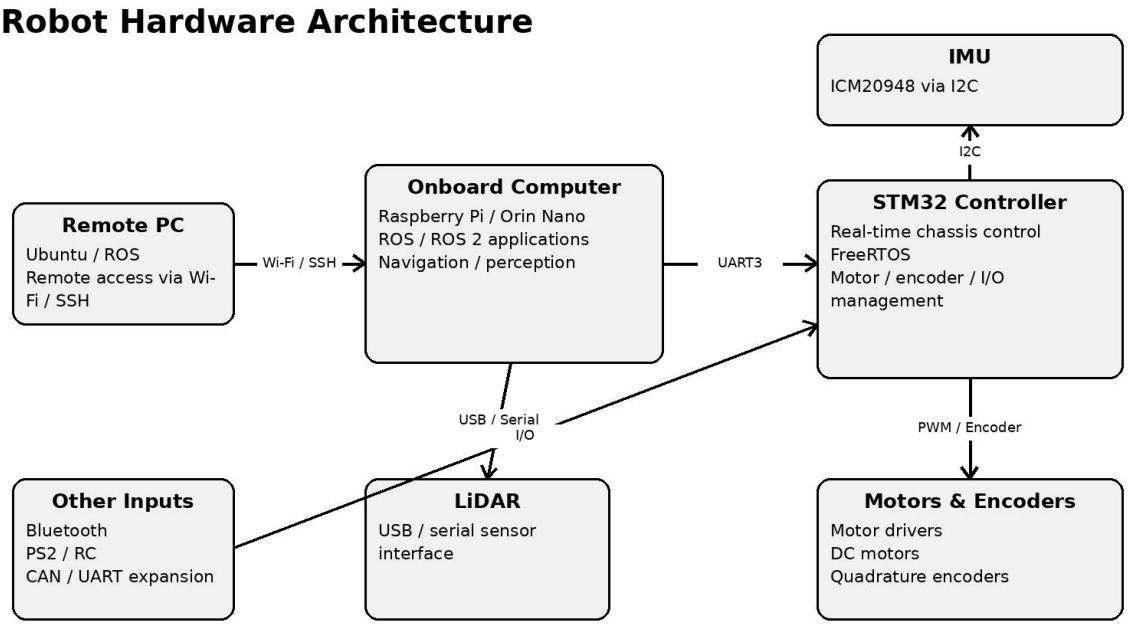


Figure 6-1. Example robot hardware architecture

LiDAR is a key sensor for navigation and environmental perception. It measures the distance and shape of surrounding objects using reflected laser light and can provide data for mapping, path planning, and obstacle avoidance.

6.2 STM32 Firmware Architecture

The controller firmware runs on the STM32F407VET6 and is developed in C using the Keil MDK-ARM (Keil 5) environment. FreeRTOS is used to support real-time multitasking.

FreeRTOS allows multiple tasks such as sensor acquisition, motion control, status display, communication, and peripheral handling to be scheduled within the same controller. This improves resource management, response time, and system organization for a mobile robot that must perform several operations concurrently.

Task scheduling is priority-based. The order in which tasks are shown in a conceptual diagram does not necessarily represent their actual priority. Interrupts remain higher priority than normal RTOS tasks. For example, when a Bluetooth/APP serial interrupt or a host-computer serial interrupt is detected, the controller responds immediately so that communication remains timely.

At power-up, the firmware initializes the chassis parameters according to the selected Car_Mode, which is derived from the chassis-selection potentiometer. Car_Mode affects the initialization and parameter selection for different chassis variants. Further source-code explanation is available in the companion STM32 firmware materials.

6.3 Motor Control Flow

All supported control modes ultimately change the target chassis velocity. The target velocity is transformed by the chassis kinematic model into individual wheel targets. The wheel-speed PI controllers then use encoder feedback to calculate motor PWM outputs.

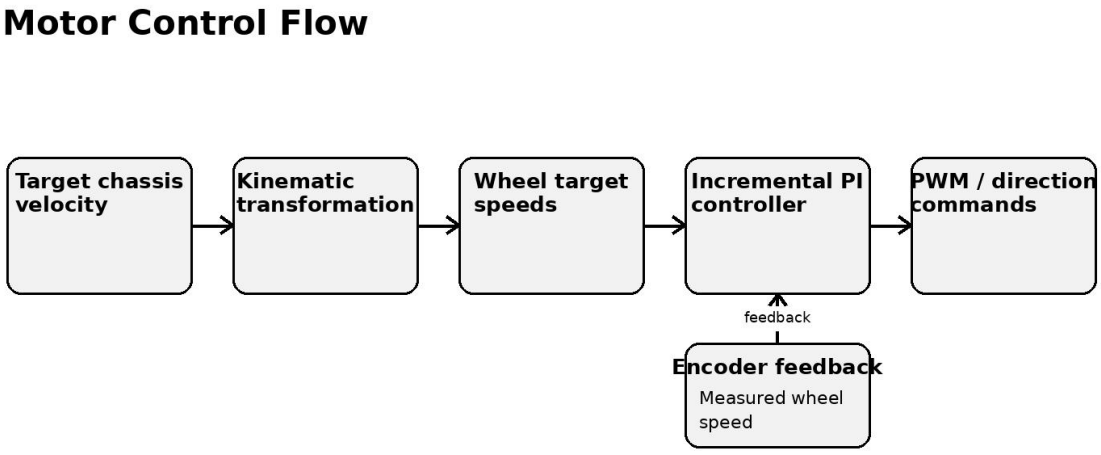


Figure 6-2. Simplified motor control flow

The exact motor-drive implementation varies by controller/chassis revision: some configurations use separate dual-channel drive boards, while others integrate the drive stage with the main controller board.

7. Firmware Programming

The STM32 controller can be programmed through a serial/USB programming path or through the SWD interface. The appropriate programming software, USB/serial driver, and firmware image are

provided as part of the development package. Hardware-interface details can vary by controller revision.

7.1 Serial/USB Programming

Hardware Preparation

Prepare the STM32 controller and the USB data cable supported by the controller revision. Depending on the board/interface revision, the programming connection may use Micro-USB or Type-C. Use the physical programming port supplied on the actual controller.

Software Preparation

Install the FlyMCU programming utility and the driver for the USB-to-TTL interface used by the controller (CH9102 in the documented configuration). After installation, confirm in the operating system Device Manager that the serial device appears without an error indicator.

Programming Procedure

1. Open the FlyMCU programming utility provided with the development package.
2. Select the correct serial/COM port and the configuration required by the supplied firmware image.
3. Select the firmware file.
4. Start programming. If "run after programming" is enabled, the controller automatically executes the firmware after download completes.

7.2 SWD Programming

The controller also supports SWD programming. The documented SWD pin assignment is PA13 for SWDIO and PA14 for SWCLK.

Hardware Preparation

- STM32 controller board.
- ST-Link programmer (or a compatible J-Link where properly configured).

Software Preparation

Install the corresponding ST-Link or J-Link driver and confirm that the programmer is detected by the operating system.

Connection

Programmer	STM32 Controller
SWDIO	PA13
SWCLK	PA14
GND	GND

Programming

Use the download/program command in the development environment to write the firmware. The default project configuration is set up for ST-Link; when J-Link is used, update the MDK debug/programmer configuration accordingly.

8. Notes and Precautions

8.1 Code and RTOS Tasks

The STM32 firmware is organized around RTOS tasks rather than a single interrupt-driven control loop. Higher-priority tasks are scheduled ahead of lower-priority tasks, while hardware interrupts

remain above task-level scheduling. If one RTOS task contains an execution error, the program may become blocked at that point. For example, if a serial-transmit task is executed before the corresponding serial interface is correctly initialized, the task may prevent normal program execution. If the firmware appears to hang during debugging, inspect each task and initialization path for errors.

8.2 Power Interfaces

The controller provides 5 V and 3.3 V outputs for external devices, but the output current is limited. The recommended limits are approximately 1.5 A for the 5 V output and 200 mA for the 3.3 V output. The adapter/power board contains two 5 V supply paths: one for basic peripherals and another dedicated to the Raspberry Pi / Orin Nano class onboard computer through a USB power connector.

8.3 Motor Use and Protection

Avoid prolonged motor stall conditions, which can damage the controller or drive electronics. Before the robot has been fully tested, support the chassis so the wheels are off the ground. This reduces the risk of unintended motion during software debugging.

8.4 Battery Use and Maintenance

Battery voltage is displayed on the chassis status display. When the voltage falls below approximately 20 V, charge the battery promptly. The battery includes over-discharge and over-charge protection. Do not operate the robot from the battery while the battery is charging.

After charging, close the charging-port cover to reduce the risk of accidental contact or short circuit.

End of Manual